量化推理与显存预算

摘要：本文讨论在显存受限的设备上做推理加速的取舍。我们把可用的杠杆分成四类：数值精度、批大小、输入长度与算子实现。四者的收益量级、质量风险与实现成本差别很大，因此顺序比力度更重要。

一章　先定位瓶颈。加速之前必须先回答“慢在哪里”。若设备功耗长期偏低而占用接近满载，说明瓶颈在显存带宽而不是算力，此时降低数值精度往往立竿见影；反之若功耗已接近上限，则说明算力受限，精度的收益会小得多。

二章　半精度。半精度把显存带宽需求减半因此对带宽受限的设备收益最大。它的实现成本极低，通常只需一次类型转换，而且对排序类任务几乎不改变结果——实测打分差异在千分之几的量级，前几名顺序完全不变。因此在带宽受限的笔记本级设备上，它是首选杠杆。

三章　半精度的风险。需要留意的是溢出与下溢：某些归一化操作在半精度下会损失有效位，表现为极少数样本的分数异常。上线前应当在评测集上逐条比对半精度与全精度的排序结果，把差异量化成可复核的数字，而不是凭抽样观察下结论。

四章　整数量化。八位整数量化对逐元素运算友好但对注意力结构更敏感。逐元素运算与矩阵乘法对量化误差不敏感，而注意力里的缩放与归一化会把误差放大。因此是否采用整数量化取决于模型结构，交叉编码器这类结构需要更谨慎的校准与更充分的回归测试。

五章　权重与激活。权重量化相对简单，因为权重分布在推理前已知；激活量化需要校准数据来估计动态范围，一旦实际输入分布偏离校准集，误差会显著上升。在单用户本地场景里输入分布很难覆盖，这是整数量化的主要风险来源。

六章　批大小。批大小对延迟的影响在小批量区间基本平坦。原因是小批量时设备已经无法填满，增大批量不会带来额外收益；只有在批量足够大、算力真正成为瓶颈之后，吞吐才会随批量上升。因此把批量作为延迟优化手段通常是无效的。

七章　输入长度。截断只在能砍掉长尾时才值得因为它同时丢掉了上下文。注意力计算量随长度平方增长，因此把明显超长的输入截断收益很大；但实测发现，如果截断只影响少数样本，整体收益会远小于预期，因为它并没有降低批量里的最长序列长度。

八章　最长序列决定成本。批处理会把同一批内的序列补齐到最长的那一条，因此成本由最长序列而不是平均长度决定。这解释了为什么“平均长度没变但耗时下降”的现象：真正变化的是最长序列。

九章　显存预算。显存预算必须把缓存分配器保留的已释放块算进去。运行时的缓存分配器会保留已释放的块以供复用，从外部观测到的占用因此高于模型本身的大小。在多个模型轮流加载的场景里，如果不主动清空缓存，后加载的模型会面临更差的显存条件甚至直接失败。

十章　模型生命周期。同时加载嵌入模型与重排模型会占用两份权重。若两者不是每步都需要，应当在阶段之间显式释放，而不是依赖垃圾回收；实测显存峰值可以从接近上限降到不足一半。

十一章　加速与质量必须同时报告。推理加速档位必须与质量指标一起报告否则数字无法解释。只报延迟会诱导不断加码截断与量化，直到某天发现质量已经悄悄下降到不可接受。正确做法是把质量指标与延迟写在同一份报告里，并把档位作为口径的一部分记录。

十二章　可回退性。任何加速档位都必须能一键回到改动前的行为，且回退不需要重新构建。质量异常往往在真实使用中才被发现，此时留给处理的时间是分钟级，而不是一次发版的时间。

十三章　冷启动。首次调用需要加载权重与初始化上下文，耗时远高于稳态。在交互场景里这次冷启动会被用户直接感知为“第一次提问特别慢”，因此应当在与用户无关的时刻预热，而不是等第一次请求触发。

十四章　与批量的配合。批量推理提升吞吐的前提是请求成批到达；单用户场景下请求稀疏，强行攒批只会把每个请求都拖到批窗口结束才执行。因此攒批窗口必须很短，或者干脆只在批任务上启用。

十五章　精度回归的做法。加速档位上线前应当在同一份评测上做双臂对照：同用例集、同机器、同

候选参数，只改精度或截断。判定标准不是“看起来差不多”，而是排序结果与全精度基线的逐条差异是否在既定范围内。

十六章　显存不足的表现。显存耗尽时框架抛出的往往是分配失败，但真实原因可能是碎片化而不是容量不足。两者的修复方式不同：前者要减小批量或输入长度，后者要靠阶段之间的显式释放来整理。

１８　延迟的分位数。延迟指标必须报分位数因为用户感知的是尾部。平均值会被大量快速请求拉低，而少数极慢请求才是用户抱怨的来源。因此加速效果应当用第九十五或第九十九百分位衡量，只看均值会得出「已经很快了」的错误结论。

１９　尾部的来源。极慢请求通常来自特别长的输入或首次触发的冷启动，这两类原因对应完全不同的修复手段：前者要限制输入长度，后者要预热。把尾部延迟和输入长度一起记录，才能判断该修哪一个。

２０　显存峰值的归因。显存峰值取决于模型生命周期而不是模型大小。加载顺序、释放时机、以及是否存在两个模型同时驻留，都会显著改变观测到的峰值。因此排查显存问题时先画出模型的生命周期时间轴，再谈参数与批量。

２１　显存碎片。反复加载与释放会在分配器里留下不连续的空闲块，此时总空闲量足够但无法满足一次较大的连续分配。主动清空缓存或改为固定顺序加载，通常比降低批量更有效。

２２　批量的上限。显存峰值取决于模型生命周期而不是模型大小之外，批量还受输入长度影响：批内最长序列决定了本次前向的显存占用，因此「批量　８」在不同输入分布下可能是完全不同的两件事。报告批量时必须同时报告长度分布。

２３　精度的选择。八位整数量化对逐元素运算友好但对注意力结构更敏感。在本地设备上权衡顺序应当是：先试半精度，再试截断，最后才考虑整数量化。这个顺序是按「收益　÷　实现风险」排的，而不是按理论加速比排的。

２４　量化的校准。整数量化需要一小批校准数据来估计激活范围。校准集应当来自真实使用分布，否则会出现「离线指标没变、线上输入变差」的现象。在小规模单用户场景里，这一点尤其难以满足。

２５　加速与质量的耦合。推理加速档位必须与质量指标一起报告否则数字无法解释的反面是：只报质量会掩盖成本，只报速度会掩盖退化。实验报告应当把两者放在同一张表里，并要求任何一项加速都附带质量对照的结果。

２６　基线的冻结。加速方案上线前必须先冻结基线否则收益无法归因。若基线本身在变动，那么两次实验的差值既包含加速的收益也包含基线的漂移，无法分离。因此加速实验应当先锁定基线，再逐项引入变动。

２７　双臂对照的最小规模。每引入一个变动就重跑一次全量评测是昂贵的，但只跑三五个用例又不足以判断质量是否退化。可行折中是固定一组「敏感用例子集」，它由历史退化样本构成，规模小但命中率高。

２８　回退演练。回退开关必须在真实环境里演练过一次，否则它只是纸面上的开关。演练内容至少包括：切换后是否立即生效、缓存是否随之失效、以及切换期间正在处理的请求会怎样。

２９　结论前置的一句。任何加速方案的价值都应当能被一句话说清：在什么设备上、用哪个档位、把哪一项指标从多少降到多少、代价是什么。说不清这句话的方案，通常是把多个变动混在一起的结果。

３０　结论。加速的顺序应当是：先定位瓶颈类型，再选与之匹配的杠杆，最后用同一份评测确认质量未变。把顺序颠倒过来，就会得到一组说不清收益来源、也说不清质量代价的数字。

３１　环境差异。同一份代码在笔记本与台式机上可能呈现完全不同的瓶颈：前者受功耗与带宽限制，后者受算力限制。因此加速结论必须附带设备与功耗条件，否则迁移到另一台机器后收益可能消失甚至为负。

3 2　依赖版本。推理框架的版本变化会改变默认的算子实现、线程数与内存策略，进而改变实测延迟。因此报告里应当记录框架版本，并且在升级框架之后重新测一遍基线。

3 3　预热与稳态。首次推理通常包含构图与显存分配，耗时可达成稳态的数倍。评测应当丢弃前若干次调用，或者显式预热，否则测到的是启动开销而不是推理开销。

3 4　观测的不确定性。延迟测量本身有噪声，因此在做结论前先重复三次，确认组内波动远小于组间差异。若两组差异落在波动范围内，任何加速比都是不可信的。

3 5　结论的落地形态。加速方案最终应当落到一个可切换的档位与一条回退路径上，并在报告里写清默认值与回退方式。只给出一个「更快」的实现而不给回退，等于把风险留给未来的自己。